

Posse: A Cost-Aware Operating System for Long-Horizon, Human-Steered Fleets of LLM Agents

Anonymous Author(s)

Affiliation withheld for review

Under submission

Abstract

LLM agents are being deployed at rapid scale — for software engineering, research, data analysis, and business operations — and increasingly they run not as a single supervised task but as *fleets* of agents operating continuously over long horizons. This shift exposes a problem that agent frameworks, which optimize the reasoning *inside* one task, largely leave unsolved: how to *use and manage many agents efficiently* over time. Each agent’s work must stay coherent across interruptions and restarts, and many agents must be organized and steered concurrently. The system must keep itself running and recover from failures; and, because every token is billed and long-lived agents carry large contexts, cost must be engineered as a deployment grows. We present **Posse**, an operating system for long-horizon agent fleets, organized around seven requirements (R1–R7) that session-based frameworks miss. Posse groups agents into user-defined *precincts* whose durable, bounded, auditable records form a *shared* memory substrate — a new or different agent reads them to pick up a line of work cheaply, instead of the expensive habit of keeping one agent alive for the whole horizon; runs each unit of work as a *deputy* whose identity survives process death; steers and interrupts deputies through an asynchronous message channel; and supervises the whole fleet with a single always-on *Sheriff*. Three principles carry the design: identity decoupled from process liveness, token cost treated as a first-class scheduling resource, and a zero-cost mechanical control plane separated from paid model cognition. From the last principle we derive a provider-parameterized, cache-aware scheduling policy whose poll-versus-park break-even we give in closed form. Posse is released as open source; the framework ships the Sheriff and a Receptionist front desk, and users define the precincts that fit their domain.

1 Introduction

Large language model (LLM) agents have moved rapidly from demonstrations to deployment. Coding agents open pull requests and run for hours [?, ?]; research agents autonomously gather and synthesize sources [?]; agents drive data analysis, customer operations, and multi-step business workflows. A second shift is now underway: rather than one agent invoked for one task, organizations run *fleets* of agents — many at once, over long horizons, increasingly with little human attention. A fast-growing ecosystem reflects this — meta-harnesses that orchestrate and swap between agent backends [?, ?], secured runtimes that host third-party agents [?], and observability platforms that trace and price them [?, ?].

As deployments scale from one supervised session to many long-running agents, a different problem dominates — not how a single agent *reasons*, but how to *use and manage a fleet efficiently* over time. Four practical difficulties recur:

- **Continuity.** A line of work and its follow-ups must stay coherent across interruptions, restarts, and the death of the process that ran it.
- **Concurrency.** Many independent agents must be organized, owned, and steered without collapsing into an untraceable sprawl.
- **Autonomy.** The system must keep itself running — routing work, staying alive, recovering from crashes and provider rate limits — with no human babysitting a terminal.
- **Cost.** Because every token is billed and long-lived

agents carry large contexts, cost must be engineered as the fleet grows, not discovered on the invoice.

This management layer is where real deployments strain, and it is largely outside what today’s agent frameworks address.

Why existing systems fall short. Each solves part of the picture and leaves the rest to the application. Agent frameworks [?, ?, ?, ?] optimize the *inside* of one task episode — reasoning, tool use, division of labor — and assume a human is present for its duration. Meta-harnesses [?, ?] run and swap agent backends but do not manage a long-horizon fleet’s identity, recovery, or cost. Secured runtimes [?, ?] isolate and host agents without durable cross-restart identity or a cost model. Observability tools [?, ?, ?] *watch* agents but do not *manage* them. Durable-execution engines [?, ?] give identity and resume-across-crash to ordinary workflows, but have no notion of LLM *token* cost, bounded LLM *memory*, or an absent human. No existing system covers the whole management layer (§??).

This paper. We present **Posse**¹, an operating system for long-horizon agent fleets. Posse distills the difficulties above into seven requirements (R1–R7, §??) and meets them with three principles: (I1) a thread’s identity is a durable *record*, decoupled from any live process; (I2) token cost is a first-class *scheduling* resource; and (I3) a zero-cost mechanical control plane is separated from paid model cognition. Concretely, Posse organizes work into user-defined *precincts* with durable, bounded memory;

¹The organizing metaphor is a *Sheriff* (a single autonomous system manager) supervising *Deputies* (ephemeral worker agents), grouped into *Precincts* (domains).

runs each unit of work as an ephemeral *deputy* whose identity outlives its process; steers deputies through an asynchronous message channel that can interrupt them mid-task; and supervises the whole fleet with a single always-on *Sheriff*. The framework ships only the Sheriff and a *Receptionist* front desk; users define the precincts that fit their domain.

Contributions.

1. A *requirements analysis* (R1–R7) for long-horizon, human-steered agent-fleet management, and a comparison showing that popular systems — built for supervised single episodes or for adjacent concerns — cover it only partially (§??).
2. Four design principles that resolve the tensions among the requirements (§??).
3. A modular design realizing them — precinct memory (§??), deputy continuity (§??), the control plane (§??), the Sheriff (§??), cost-aware scheduling (§??), multi-agent coordination (§??), and observability (§??) — the core modules presented with the alternatives we considered and our rationale.
4. An analytical, provider-parameterized *cost-scheduling result*: a cache-TTL break-even that makes the poll-versus-park decision optimal, given in closed form and independent of context size (§??).
5. An open-source implementation; the framework ships the Sheriff and Receptionist, and users create their own precincts.

2 Background and Requirements

2.1 The landscape

Systems that touch agent management cluster into six families.

(a) Autonomous single-agent loops. AutoGPT and BabyAGI [?, ?] wrap one LLM in a plan-act-observe loop; reasoning scaffolds — ReAct [?], Reflexion [?], Tree of Thoughts [?] — improve its decisions. GPT-Researcher [?] autonomously plans and synthesizes a research report. These target the quality of one agent’s trajectory.

(b) Multi-agent frameworks. AutoGen [?], CAMEL [?], MetaGPT/ChatDev [?, ?], and CrewAI [?] run several role-specialized agents that converse or follow a standard procedure to complete one project.

(c) Graph / workflow orchestration. LangGraph [?] and OpenAI’s Swarm [?] are construction kits: a developer wires agents into a stateful graph or hand-off routine that executes when invoked.

(d) Meta-harnesses and runtimes. A newer layer runs and secures agents rather than authoring them. Omnigent [?] orchestrates and hot-swaps agent backends (Claude Code, Codex, Cursor, custom) behind one interface, with policy and sandboxing; exo [?] is a comparable agent harness. NemoClaw [?] and OpenHands [?] host and sandbox agents with managed inference and a control

surface. These make agents *runnable and safe* but treat each run as transient.

(e) Observability. Langfuse [?], AgentOps [?], and Arize Phoenix [?] trace, evaluate, and price agent runs. They *observe*; they do not *drive* the fleet.

(f) Durability and OS abstractions. Durable-execution engines — Temporal [?], AWS Step Functions [?], Airflow [?] — give ordinary workflows an identity that survives process death and resumes on an event. AIOS [?] proposes an “LLM agent operating system” kernel; MemGPT [?] manages context like virtual memory; server-side threads [?] persist a conversation’s state. These are the closest in spirit to Posse and we return to them in §??.

2.2 Requirements

From the difficulties in §1 plus the operational reality of a metered API, a finite model window, and a human who cannot watch every agent, we distill seven requirements for managing a long-horizon agent fleet.

R1 Long-horizon continuous operation. Run unattended for days to weeks, not for one session.

R2 Durable thread identity & continuity. A line of work keeps one identity and its context *across process death*.

R3 Concurrent fleet management. Many independent threads, each isolated, owned, and navigable.

R4 Asynchronous human steering. A human directs and *interrupts* agents asynchronously, without holding a session open.

R5 Autonomous fault recovery. Self-heal from crashes and provider rate limits.

R6 Cost-aware scheduling. Treat token/\$ cost as a scheduled resource, not an afterthought.

R7 Bounded, shared, faithful memory. What the fleet learns must outlive the process that learned it, be readable by *any* agent, fit the model window when recalled, and come back as written, with its source — not as an approximate reconstruction the agent must second-guess.

These are not fully orthogonal: R2, R3, and R7 are facets of one *durable-record substrate* (identity, ownership, shared memory), and R1 is unreachable without R5. We list them separately because prior systems fail them *independently* — a system can manage many threads (R3) yet lose each one’s continuity on restart (R2).

2.3 The gap, requirement by requirement

The families above are strong in their own regimes, and several already meet R3’s concurrency half — many units of work at once, though rarely as isolated, owned, navigable threads. The rest of the list goes largely unmet, for reasons that split by family. The families that build, run, or orchestrate agents treat each run as a self-contained episode — one task, one run, one project — with a human close at hand whenever input is needed; observability

platforms are read-only by design; and durable-execution engines run for months unattended, but over conventional code, where windows, contexts, and token bills do not exist. Five failures follow.

Identity dies with the process. In most systems a “thread” is a running object — a process, a graph run, a chat — so when the object ends, the thread’s identity and accumulated context end with it (fails R2). The loss is not exotic. An analyst has an agent draft a report on Monday and on Thursday replies, “now break it down by region”; real work arrives exactly this way, as a sequence of follow-ups. But if Monday’s process has exited, Thursday’s reply lands on a *fresh* agent that knows nothing of the data sources, caveats, and choices behind the draft, and the human must re-brief it with a hand-written summary that is inevitably lossy. AutoGPT [?], CrewAI [?], and AutoGen [?] tie the thread to a run that ends with the task. LangGraph [?] can persist a run’s state, but only if the application holds the `thread_id` and reloads it — nothing routes a reply that arrives days later back to that state on its own. Server-side threads [?] persist the conversation itself, and durable-execution engines [?, ?] the workflow — but a persisted transcript is not a routed thread: nothing watches for Thursday’s reply, matches it to Monday’s thread, and relaunches an agent on it.

What an agent learns is trapped in its session. The same coupling leaves knowledge stranded along with identity. Suppose agent A spends a long session learning a domain’s quirks: which tools are reliable, which apparent shortcuts dead-end, what was tried and rejected. The next task in that domain gets a new agent that knows none of it, and existing designs offer only unattractive ways to pass the knowledge on. Replaying A’s transcript violates R7’s bounded-recall clause — and presumes the transcript was persisted at all. Keeping A alive for the whole horizon violates it too: one context growing without bound, the very pathology a finite window and a metered bill forbid. Bolting retrieval onto an external store scales, but what it returns is whichever fragments similarity search surfaces — the lesson comes back piecemeal, never guaranteed whole, not as it was written. What a domain’s agents have learned should be a shared, reusable asset that any newcomer picks up cheaply and can rely on. MemGPT [?] comes closest by bounding a single agent’s context, and AIOS [?] by managing per-agent memory in its kernel, but per-agent memory is still trapped memory — neither turns what one agent learned into records *any* agent can read (each meets R7 only in part).

Steering assumes a waiting human. In a fleet the human is absent by design, but their feedback cannot wait for the next session. An agent forty minutes into an expensive job may be working from a specification the human has since realized is wrong; the correction is useful only if it reaches *that* agent promptly and redirects it *without* discarding the forty minutes of computation already done. Existing designs invert the relationship. LangGraph’s `interrupt()` pauses the run at a predeter-

mined point and *waits* for the human to supply input — the graph, not the human, decides when feedback may arrive. AutoGen likewise solicits input inline, through a human proxy. In both, a correction arriving from outside the script has no path into a run in flight; the operator’s recourse is to stop the job and restart it corrected — forfeiting the very computation the correction was meant to save. Temporal narrows the gap: a signal is durable and needs no live worker process. But it must name a still-open workflow execution [?], and what it updates is the program’s state — redirecting an agent means reaching its *reasoning*, folding the correction into a context mid-flight, which no workflow signal models (meets R4 for code, not for agents).

Nothing keeps the fleet alive. Over a horizon of weeks, the routine failures of a shared, pay-per-token deployment are a certainty rather than a tail risk: the provider’s weekly rate limit trips at 2 a.m., the machine reboots, a process is OOM-killed. Without recovery, the affected work silently stalls until a human notices — perhaps not until morning — and a fleet that needs a minder is not autonomous in any useful sense. Agent frameworks retry a failed tool *call* inside a run — LangGraph attaches retry policies to graph nodes [?] — and durable-execution engines [?, ?] restart crashed *workflows*. Secured runtimes [?, ?] host the agent’s process and will restart a dead container, but leave rate-limit-aware detection and timing to the application. None treats the agent’s *process* as the thing to supervise: noticing that it has died or been rate-limit-blocked, then relaunching it — immediately after a crash, at the reset after a limit. Durable engines earn their R1 and R5 for what they run; nothing yet extends either to the agent’s own process.

Cost is observed, never scheduled. At fleet scale the dominant cost is *touching* large contexts, and whether each touch is a warm cached read or a cold full-context rewrite is decided by scheduling rather than by model choice. Consider a fleet whose agents each wait on hour-long jobs, checking progress every few minutes: a check that lands after the short cache window rewrites the *entire* context at roughly 12.5× the price of a cached read (one major provider’s published caching prices [?]), so a cadence only slightly too slow can quietly multiply that fleet’s monthly bill by an order of magnitude. Observability tools [?, ?] *measure* this spend after the fact, and AIOS [?] schedules LLM calls for latency rather than for tokens, but no orchestration framework we surveyed *schedules* the poll-versus-park decision — keep the context warm, or sleep until an event wakes the agent — against the provider’s cache economics (fails R6; §?? derives the break-even).

Table ?? maps where each design puts its effort. Durable-execution engines come closest on identity and recovery, but what they make durable is *compute* — code state, not an agent’s context — and their model has no notion of a token bill. The gap Posse targets is the *conjunction*: asynchronous steering (R4), scheduled cost

Table 1. Design emphasis in the long-horizon fleet-management regime of §?? — *not* a quality ranking; each system excels in the regime it targets. Rubric: ✓ = a first-class system property; ◦ = partial, or present but left to the application; × = not a concern of the design. Cells reflect each system’s *default, first-class* stance, not what is achievable with custom code. “Cost *sched.*” is cost-aware *scheduling*, distinct from the cost *tracking* that observability tools do well.

	Type	R1	R2	R3	R4	R5	R6	R7
		horizon	identity	fleet	async H-in-loop	recovery	cost sched.	LLM memory
AutoGPT / BabyAGI [?, ?]	auton. loop	◦	×	×	×	×	×	◦
ReAct / Reflexion [?, ?]	reasoning loop	×	×	×	×	×	×	◦
GPT-Researcher [?]	research agent	◦	×	×	×	×	×	◦
AutoGen [?]	multi-agent chat	×	×	◦	◦	×	×	×
CrewAI / MetaGPT [?, ?]	role crew / SOP	×	×	◦	×	×	×	×
LangGraph [?]	graph orchestration	◦	◦	◦	◦	◦	×	◦
Omnigent [?] / exo [?]	meta-harness	◦	◦	✓	◦	◦	×	×
NemoClaw [?] / OpenHands [?]	secured runtime	◦	◦	◦	◦	◦	×	◦
Langfuse / AgentOps [?, ?]	observability	×	×	◦	×	×	◦	×
Durable exec. (Temporal) [?, ?]	workflow engine	✓	✓	✓	◦	✓	×	×
AIOS / MemGPT [?, ?]	agent OS / memory OS	◦	◦	◦	×	◦	◦	◦
Posse (this work)	agent ops	✓	✓	✓	✓	✓	✓	✓

(R6), and shared, faithful memory (R7), on a substrate that gives agent threads durable identity (R2), ownership and navigability at fleet scale (R3), and process-level supervision (R1, R5) — while reusing the reasoning and multi-agent ideas of families (a)–(c) *inside* its deputies, the harness/runtime ideas of (d) to run them, observability’s read-only view (e) as its dashboard, and the OS/memory stance of (f) in its precinct records. One ✓ in the Posse row warrants a word. On R2 the design hands the user a *continuity dial* per *case*, Posse’s term for one thread of work; the dial currently has two settings. Replying to a case, even a finished one, relaunches its original deputy on its recorded session, identity and context intact. Opening a new case gets a fresh deputy briefed from the durable records. Only the cost-aware policy that would set the dial automatically remains future work (§??).

3 Design Overview

3.1 Four principles

The requirements pull against each other. Full continuity (R2) wants one long-lived context; cost (R6) and the finite window punish exactly that. Autonomy (R1, R5) wants always-on processes; always-on-ness is what a naive design bills you for. Shared knowledge (R7) wants to accumulate inside one agent’s context, where it is neither durable nor legible to a second agent. Posse rests on four decisions.

I1. Decouple identity from liveness. A thread’s identity is a *durable record*, not a running process. Continuity (R2) is provided by persisted per-thread records and a lineage map, so a thread can be resumed — or rebuilt into a fresh process under the same identity — long after its original process died.

I2. Make token cost a first-class scheduling resource. Every waiting decision (poll vs. sleep), memory decision (keep vs. compact), and model choice (cheap vs. capable) is taken against an explicit cost model derived from the provider’s cache economics (R6). Cost is scheduled, not discovered on the invoice.

I3. Separate zero-cost mechanical control from paid cognition. The control plane — routing, liveness, recovery, waking, resource serialization, compaction *triggers* — makes *no* LLM calls. The model is invoked only for actual work and a handful of manager decisions. This is what makes always-on operation (R1) affordable (R6).

I4. Treat memory as a shared record, not a private context. Knowledge that a naive design traps inside one long-lived context — legible only while that process, and its bill, stays alive — is instead externalized into per-precinct durable records (R7): a compacted ledger, an append-only case log, and per-case files. Every deputy reads them at spawn, so handing context to a second agent costs a file read rather than a kept-alive session. Continuity becomes a property of the *precinct*, not of any one agent: any deputy can pick up where another left off.

3.2 System overview

Figure ?? shows the result. Work is organized into **precincts** — user-defined domains (e.g. evaluation, development, research, operations). The framework ships only a *Receptionist* front desk and the *Sheriff*; a user creates whatever precincts fit their work. Each precinct owns durable *records*: a compacted *ledger* (big-picture digest), an append-only *case log* (index of finished work), and per-thread *case files* (§??). We distinguish a *thread* — the durable identity of a line of work — from a *case* — one numbered unit of work on it; a thread is a sequence of cases. Each case is worked by a **deputy**: an ephemeral

LLM worker that lives for one case, messages the human a plan/milestones/result, and closes the case by appending to the records (§??).

The human steers through an **asynchronous message channel** (our implementation uses email; Fig. ??, top). An always-on *router* maps each message — mechanically, no LLM — to an existing thread (a reply), a new deputy in a tagged precinct, or the Receptionist (§??). Overseeing everything is the **Sheriff** (left): a single system manager that supervises deputy liveness and precinct-records health. Its monitoring is zero-API (I3); it spends tokens only to compact a ledger or decide a deputy’s change-request. Underneath sit always-on daemons that route, relaunch, event-wake, and serialize contended resources; a read-only **dashboard** (right) observes it all. The “operating system” framing is literal in the classic-services sense: the cost-aware wait policy (§??) is the *scheduler*, ledger compaction (§??) is memory *paging*, the active-deputies board is the *process table*, and the Sheriff is the process *supervisor* — a mapping AIOS [?] argues for in the abstract and Posse supplies as a running system.

3.3 Life of a case

Figure ?? traces one case. A message arrives and is routed; a deputy is spawned or an existing identity resumed. At spawn the deputy is handed its precinct’s *ledger* — injected into its opening context — so it starts already knowing the domain’s big picture and the state of related work (I4), rather than rediscovering it. The deputy messages a plan, works in small chunks, offloads compute to a detached job, and — for a long job — *submits it to a job manager and sleeps*, to be woken by an event rather than by polling (§??). If the human replies mid-case, the deputy is *surgically interrupted*: its process is killed and relaunched with the new message injected, while its detached jobs keep running (§??); on resume it reads its mailbox *first*. Finally it reports a result and *closes* the case: one ledger paragraph, one case-log line, and a done-marker. Continuity (the dashed band) is carried not by the process — which may have died and been replaced — but by the durable case file and a resume-or-reinject decision (§??). Underneath, the Sheriff and daemons keep the deputy alive and wake it, with no LLM call.

The remaining sections detail each module: what it does, the alternatives, and why we chose as we did.

4 Precincts and Bounded Memory

Role (R7, R2, R3). A precinct is a user-defined domain with durable, structured memory. Its three record types form a hierarchy from most- to least-compacted: a *ledger* (a bounded big-picture digest, rewritten by compaction as it grows), an append-only *case log* (one line per finished case, never rewritten, so attribution stays honest), and *case files* (the full per-thread record, the source of truth a resumed thread reads). All access goes through one concurrency-safe records manager, never raw files —

the deliberate choke point that enforces the append-only contract: a deputy may only *append* (one ledger paragraph and one case-log line, at close); only the Sheriff may compact, retract, or create/delete a precinct.

Cheap memory sharing, without a long-lived agent. This is the payoff of externalizing memory into precinct records. In the usual pattern, knowledge accrues *inside* one agent’s long session: a *different* agent cannot reuse it, and keeping that agent alive for the whole horizon is expensive (its context grows without bound). Posse decouples the two. A deputy may still take a sequence of follow-up cases (the classic “one long session”), but that is no longer the *only* carrier of knowledge: because every case’s outcome is distilled into the shared ledger, log, and case files, *any* agent — the same deputy on a follow-up, or a brand-new one, or one in another concurrent thread — acquires the precinct’s accumulated context by *reading* it. Knowledge transfer costs a bounded read, not a kept-alive session — turning “share what this agent learned” from an ad-hoc, expensive operation into a first-class, cheap one (R7, R2, and the cost side of R6).

Framework ships empty of domains. Only the Sheriff and Receptionist are built in. A user creates precincts for their own work — evaluation, development, research, whatever fits — each with its own memory and default model. Precincts are thus the unit of both *organization* and *extension*.

Design alternatives for long-term memory. The crux is R7 versus the model window and R6:

1. *One growing context per domain.* Maximal continuity, but the context grows without bound, eventually exceeds the window, and — per I2 — becomes the most expensive thing to touch.
2. *Per-episode memory only.* Cheap and bounded, but forgets between cases; every follow-up is re-briefed by hand (fails R2, R7).
3. *Retrieval over a vector store.* Scales, but retrieval is lossy and hard for a human to audit (“what does this domain know?”).
4. **Chosen: a compacted ledger + append-only log + full case files**, managed by a separate zero-cost process. The ledger stays within a token budget (bounded, cheap to load); nothing is silently lost, because the log and case files are never compacted away; and the whole thing is plain text a human can read. Compaction (an LLM “reflection” over the domain, §??) is a coarse, operations-level analog of MemGPT’s paging [?] and Generative Agents’ reflection [?], but performed by the manager, not the worker, at domain granularity.

Each precinct also carries a *default model* (a capable model for most work, a cheaper-but-capable model for token-heavy domains), so cost/capability tiering is a per-domain policy (§??).

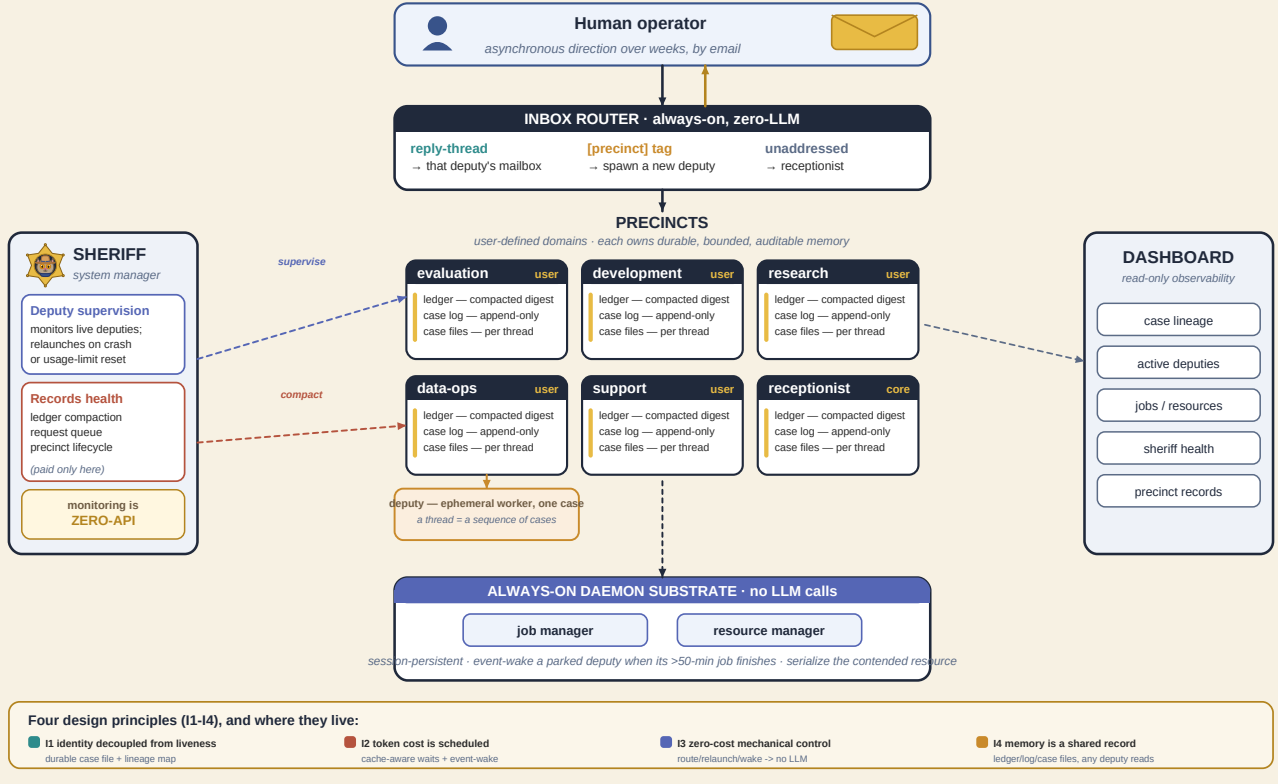


Figure 1. The Posse architecture. Work flows down the center (human → message channel → router → user-defined precincts → deputies), with the always-on daemon substrate underneath. The *Sheriff* (left) manages the fleet with zero-API monitoring, spending tokens only on ledger compaction and change-request decisions; the *dashboard* (right) observes read-only. The framework ships only the Sheriff and Receptionist; the other precincts are illustrative and user-created.

5 Deputies and Thread Continuity

Role (R2). A deputy is an ordinary tool-using LLM agent that owns *one case*. The hard part is not what it does but how its *thread* survives everything that happens to the process running it.

The failure to avoid. The tempting design ties a thread’s identity to its process: a worker that finishes ends its session, so the next follow-up cannot find it alive and spawns a brand-new worker with a fresh name and blank context, re-briefed by a hand-written spec. Iterated over a fleet, this accumulates one throwaway identity per task and scatters a single project across many differently-named agents — continuity tied to *process liveness* rather than to the work.

The principle (I1). Route every follow-up to a *stable, meaningful identity* recorded in a persisted map — not to whatever process happens to be alive. Continue that identity by *context budget*: while its context is cheap, **resume** the same session (full live context, warm cache); once it grows past budget, spawn a fresh process under the *same* identity and inject a compact brief (prior case numbers, keywords, pointers to the prior transcript and deliverables). Identity is stable either way; context stays bounded; the expensive cold revive of a giant context is avoided.

Design alternatives (continuity policy). We considered five points on the continuity–cost spectrum:

- Reuse-if-alive, else new agent.* Simplest and naturally parallel, but continuity is accidental and naming is chaotic.
- Standing domain agents* (never die). Strong continuity, one identity per domain, but one unbounded context: window pressure, cross-task interference, serialization.
- Per-thread persistent agent* (resume the exact session on demand). Precise continuity and a stable identity, but inherits B’s unbounded growth on long threads.
- Fresh agent per task + injected context.* Always bounded and cheap, lineage explicit, but the injected brief is a summary, not the live session, and naming multiplies unless paired with a stable name.
- Chosen direction: hybrid.** A stable per-thread *name* (like C/D) plus a *resume-if-cheap-else-reinject* switch (C when cheap, D when not), with the map persisted at creation. Among the five, only E avoids a *severe* weakness on continuity, identity, and cost.



Life of a case

one deputy = a single linear LLM context (one case of a thread)

the deputy emails the human at each step (plan · milestones · FINAL) and drains its mailbox first on resume

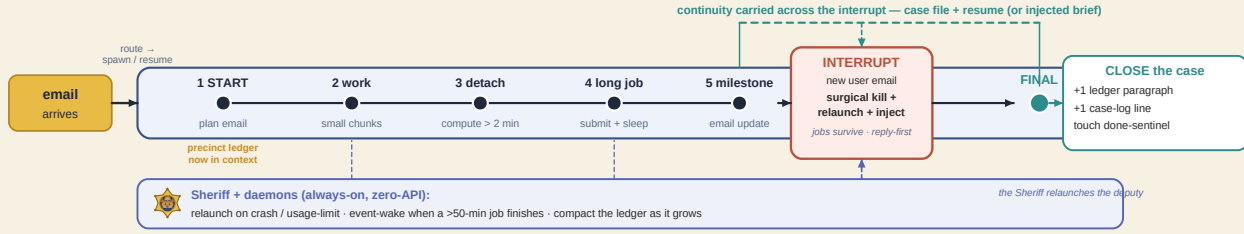


Figure 2. Life of a case. A deputy is one linear LLM context. Continuity survives the interrupt (top dashed band) via the durable case file plus session resume or an injected brief; the always-on Sheriff/daemon layer (bottom) relaunched, event-wakes, and compacts — with no LLM calls.

Deployment status (honest accounting). The *durable* half of E is shipped: a persisted lineage map records each thread’s identity at creation, and an active-deputies board makes the current owner authoritative. The *context-budget switch* — metering context size to choose resume-vs-reinject automatically — is designed but only *partly realized*: today the router resumes a thread’s recorded session when valid and otherwise spawns fresh under a routed identity, without yet measuring context size to drive the choice. E is our chosen direction and its identity substrate is live; the automatic cost-threshold is future work.

6 The Control Plane

Role (R4). The human steers the fleet through an *asynchronous message channel*, which is also the *interrupt* mechanism. We chose an ordinary mailbox (email in our implementation) deliberately: it is asynchronous by nature (the human is never blocked and never blocking), durable (the mailbox is the queue), it threads (a reply carries its lineage), and it needs no bespoke client.

Routing. An always-on router maps each message to its destination *mechanically* — zero-LLM (I3) — on two signals: (1) a reply threaded to a deputy’s earlier message → that deputy’s *mailbox*; (2) an explicit [precinct] tag → a new deputy in that precinct (a reply to an already-*dead* deputy is attributed to its precinct via the persisted lineage map). Only genuinely *unaddressed* messages reach the Receptionist, whose disposition (answer, or write a spec and spawn a deputy) is the one routing path that spends an LLM turn. An earlier design scored message text against all agents with a keyword matcher; we removed it after it mis-routed under ambiguity. Deterministic routing is cheaper and more predictable.

Interruption. When a reply targets a *live* deputy, the message is appended to its mailbox under a lock and the deputy is *surgically interrupted*: only its LLM process is killed and immediately relaunched with the mailbox injected; its detached compute jobs — separate processes

— keep running and are reconciled on resume. On *any* resume the deputy’s first action is to drain its mailbox and reply (the “reply-first” rule), so human feedback is never processed on a stale plan. The delivery guarantee is *at-least-once with an idempotent drain*: the append and a boot marker are flock-serialized, so a burst of replies is drained *together* on the next boot rather than lost or double-processed, and a job that finishes *during* the interrupt window is reconciled from its result file on resume, never re-submitted.

Design alternatives.

- *Kill everything vs. surgical kill.* Killing the whole session would kill in-flight compute jobs (potentially hours of work). We kill only the agent process and let jobs survive as reconcilable orphans — the single most important robustness property of the interrupt path.
- *Debounce vs. immediate.* We interrupt *immediately* on every human message, with boot-marker coalescing so a burst is drained together. Feedback latency matters more than avoiding a few extra relaunches.
- *Prompt via argv vs. stdin.* One subtle failure mode illustrates the risk: if the resume prompt is embedded in the process command line, a broad process-kill signal can match the agent’s own argv and cause a self-kill; passing prompts via *stdin* avoids it. The control plane’s own mechanics become a fault source at scale.

7 The Sheriff: Autonomous System Management

Role (R1, R5, R7). The Sheriff is the fleet’s single “manager” *role*, presented as one but implemented as two cooperating always-on loops: a *deputy-supervision* loop (liveness monitoring — crash and rate-limit recovery) and a records daemon (precinct-records health — ledger compaction, a change-request queue, precinct lifecycle). We unify the *model* (one manager, one dashboard panel)

while keeping the two proven loops physically separate; folding them into one process is a deferred option.

The hard guarantee (I3). The Sheriff’s *monitoring is zero-API*. It scans markers, process liveness, and ledger token-lengths mechanically; it makes an LLM call *only* to (a) compact a ledger that crossed its soft limit, or (b) approve/deny a deputy’s change-request. An idle pass costs nothing. This is what lets the manager run forever without being a cost center.

Compaction. When a ledger crosses a soft token budget the Sheriff compacts it toward a target with one LLM call on a cheap model, given whole-system context (all precincts’ ledgers) and a fixed prompt that forbids inventing or silently dropping a case. On any failure or rate limit it falls back to a mechanical truncation, so the ledger is *always* bounded and the daemon never crashes. Compaction is the system’s “reflection” step — performed by the manager, off the worker’s critical path.

Change-requests & authority. Records mutations that are not simple appends (retract a log line, edit a ledger, allocate an identifier, create/delete a precinct) are *sheriff-only*. A deputy requests them via a queue; the Sheriff verifies the requester by matching its (deputy, session) against a live registry — the session token minted at spawn *is* the shared secret, so a forged deputy is rejected *before* any LLM call — then decides with one LLM call returning strict JSON, performs the op, and journals it. A failed or rate-limited decision *defers*: it never auto-approves and never deletes by default. Destructive operations add gates: deleting a precinct requires no active deputies, a confirmation reply matched *mechanically* by the zero-API daemon, and a recoverable trash window before hard purge.

Design alternatives.

- *Present the liveness loop as one facet of the Sheriff, or as a separate service?* We present it as part of the single manager but keep the battle-tested liveness loop physically separate, because the relaunch logic is the most safety-critical code in the system. Unifying the model without rewriting the mechanism captured the clarity at near-zero risk.
- *Mechanical vs. LLM compaction.* We use LLM-by-default with a mechanical fallback — quality when possible, safety always.
- *Who may edit records?* Concentrating all non-append authority in one identity, gated by verified requests and a journal, keeps the record trustworthy; letting deputies edit their own history would destroy attribution.

8 Cost-Aware Scheduling

Role (R6). This is where Posse most departs from prior systems: *token cost is scheduled*, against a model of the provider’s cache economics.

The cost model. Long-running deputies wait on long jobs. The naive policy — poll the result on a coarse cadence, re-reading a large context each time — is ru-

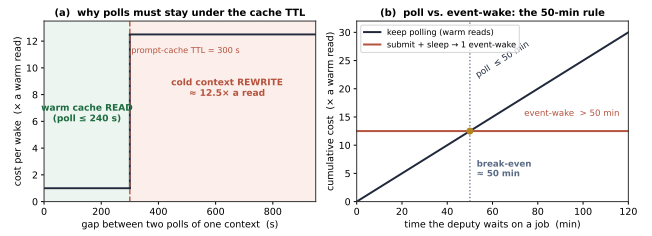


Figure 3. The cost model behind Posse’s wait policy (a *schematic* of the economics, not a trace of measured traffic). (a) A poll within the cache TTL is a warm read; past it, a cold rewrite at $\sim 12.5\times$. (b) Cumulative cost of waiting: repeated warm polls grow with duration, one event-wake is a single cold rewrite; they cross at ~ 50 min — the threshold between the two strategies.

inous, and the reason is caching, not the number of polls. Providers cache the prompt prefix with a short time-to-live (TTL): a touch within the TTL is a warm *read*; a touch after is a cold *rewrite* of the whole context, at roughly $m \approx 12.5\times$ the read (Fig. ??a). A coarse cadence makes every poll a cold rewrite of a growing context. Our analytical model (§??, Fig. ??) predicts that a coarse cadence inflates a fleet’s aggregate spend $\sim 8.9\times$; in an unmanaged deployment we observed, a policy of this kind produced a cost regression consistent with that prediction, until the scheduler below replaced it.

The policy. Two rules fall out. (1) **Short waits: poll under the TTL.** Sleep in chunks shorter than the TTL so every wake is a warm read. (2) **Long waits: submit and sleep.** Hand the wait to a zero-API job manager and *end the turn*; the manager wakes the deputy by *event* when the job finishes — one cold rewrite, once — rather than paying a warm read every few minutes for an hour.

The break-even, in closed form. Under three assumptions — (a) each poll is itself a cache-touch that renews the TTL, and the cadence is set below the TTL (e.g. ≤ 240 s under a 300 s TTL, the margin absorbing jitter) so the *next* poll lands inside the window the current one opened; (b) an event-wake is one cold rewrite plus $O(1)$ bookkeeping; (c) cost is linear in the number of touches — the accounting is simple. Let m be the rewrite-to-read multiplier and r the warm polls per hour at that cadence. Polling for a wait of t hours costs $\approx rt$ reads; an event-wake costs $\approx m$ reads. The break-even is $t^* \approx m/r$. Crucially, *context size cancels* — both a read and a rewrite scale with it — so under this model t^* is independent of how large the thread has grown, and one threshold is correct for every deputy. For $m \approx 12.5$ and $r \approx 15$ (a poll per ~ 4 min under a 5-minute TTL), $t^* \approx 50$ min (Fig. ??b).

Two more cost levers. (i) *Zero-API control plane (I3):* every always-on daemon runs without an LLM call, so only real work and rare manager decisions are billed. (ii) *Model tiering:* cheaper-but-capable models run token-heavy domains and every Sheriff call; capable models run the rest, as a per-precinct default.

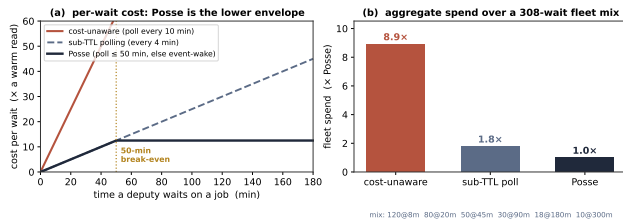


Figure 4. Analytical cost of the wait policy over a heavy-tailed fleet job-mix. (a) Per-wait cost: Posse is the lower envelope of sub-TTL polling and event-wake. (b) Aggregate fleet spend relative to Posse: a cost-unaware coarse cadence spends $\sim 8.9\times$, sub-TTL polling alone $\sim 1.8\times$. Model-derived, not measured.

Why this generalizes. The constants are provider-specific, but the principle generalizes to any backend with a *TTL-bounded* read/rewrite discount: there is a break-even m/r between “keep the context warm” and “tear down and event-wake.” (Backends with a persistent cache or a per-write surcharge change or remove it, and need the model re-derived.) Prompt-cache economics are heavily exploited at the *inference-serving* layer — prefix sharing and KV-cache reuse [?, ?, ?] and provider prompt caching [?, ?]; to our knowledge they have not been lifted into an *agent scheduler’s* wait/park decision, and we would welcome pointers to prior work.

9 Analytical Evaluation

We evaluate the cost policy *analytically* — from the model of §??, not from a benchmark against other systems (a fair one is hard across regimes, §??). Figure ?? instantiates the model for a fleet whose job-wait durations span a heavy-tailed mix (many short waits, a few multi-hour jobs). Panel (a) plots per-wait cost for three policies — a *cost-unaware* coarse cadence (every poll a cold rewrite), *sub-TTL polling* alone, and *Posse* (poll while short, event-wake past t^*). Posse is the lower envelope: it inherits sub-TTL polling’s cheap short waits and caps the long tail at a single rewrite. Panel (b) aggregates over the mix: relative to Posse, the cost-unaware policy spends $\sim 8.9\times$ and sub-TTL-polling-alone $\sim 1.8\times$ — the coarse-cadence factor being the analytical analog of the cost regression noted in §??. These are model predictions, clearly labelled as such; an empirical study — a live cost microbenchmark, a fault-injection test of the interrupt/recovery invariants (§??), and a longitudinal fleet trace — is the natural next step, and the present paper’s main threat to validity (§??).

10 Multi-Agent Coordination

Role (R3). Most cases are one deputy; four primitives handle work that is not. The orchestrator-worker and hand-off shapes are standard [?, ?]; what is specific to Posse is that they operate over *durable identities*, so a hand-off or a wait must respect the identity/continuity machinery of §??.

- **Sub-deputies.** A deputy spawns a tracked child for a sub-task, numbered as a child of the parent case, so a large effort decomposes into a *visible* tree rather than an opaque recursion.
- **Manager-wake on completion.** A parent waiting for children does not busy-wait (which would burn context, I2); it registers “wake me when X finishes” and parks, and the Sheriff wakes it when the children’s done-markers appear — the event-wake idea (§??) applied to inter-agent dependencies.
- **Joint task forces.** For work needing several agents at once, a lead plus collaborators (and an optional critic) are assembled; each slot is either a precinct (spawn a fresh deputy) or a specific deputy (a “must-take” assignment).
- **Stop, hand over, then take** (the novel rule). Assigning work to a deputy already busy on another case would interleave two live cases in one linear context. Our rule — chosen over “refuse if busy” and “switch now” — has the busy deputy hand its current case to a new deputy with full context injected (reusing §??), then take the new role as a fresh case, so a durable identity is never forced to run two live cases at once.

A throwaway *anonymous* worker — most often a critic reviewing a draft — files no records and is owned solely by its spawner, keeping the tracked record clean while enabling adversarial self-review.

11 Observability

Role (R3). A read-only dashboard renders the fleet for the human: the case-lineage forest (which case followed which), the active-deputy board, job state, Sheriff health, and each precinct’s records. It is deliberately *read-only over the same records the system runs on* — it computes nothing the system does not already persist — so it can never disagree with reality or perturb it; the persisted lineage map (§??) makes the view authoritative rather than reconstructed. This is the reconciliation / desired-state pattern of cluster controllers [?, ?], applied to an agent fleet.

12 Discussion, Related Work, and Limitations

Positioning. Posse borrows freely: its deputies are ReAct-style tool-agents [?]; its coordination echoes role-based multi-agent work [?, ?, ?]; its ledger compaction is a coarse cousin of MemGPT’s paging [?] and Generative Agents’ reflection [?]; and its “LLM-as-managed-process” stance is the one AIOS [?] argues for. Its *systems* lineage is older still: the Sheriff’s supervise-and-relaunch loop is a supervision tree in the Erlang/OTP “let it crash” tradition [?, ?]; the read-only dashboard is a reconciliation / self-healing-controller pattern [?, ?]; and durable identity that survives process death is what durable-execution engines [?, ?] (and, more weakly, DAG schedulers like Airflow [?]) give ordinary workflows. The newer har-

ness/runtime layer is complementary, not competing: a meta-harness like Omnigent [?] or exo [?] is exactly the kind of backend a deputy could run *inside*, and a secured runtime like NemoClaw [?] or OpenHands [?] is where a deputy’s process could be sandboxed; Posse is the layer *above* them that gives the fleet durable identity, async human steering, self-recovery, and cost scheduling. Job schedulers (Slurm [?]) and distributed actor runtimes (Ray [?]) likewise submit, wait on, and event-complete long tasks, but they schedule *compute* without a token-cost model or a durable LLM-thread identity. Observability platforms [?, ?, ?] are the read-only counterpart to Posse’s dashboard — Posse additionally *acts* on what it sees. The contribution we could not place in prior work is on the *cost* axis: lifting the inference layer’s cache-TTL economics [?, ?] (§??) into an agent scheduler’s wait/park decision.

Failure modes and open risks. Four risks are inherent to the design. (i) *The Sheriff is a single point of failure* — it is a root daemon restarted by an idempotent supervisor, not itself watched by a peer; a silently-dead Sheriff halts compaction and recovery until a human notices. (ii) *Unsupervised work quality*. Posse keeps deputies alive and cheap; it does not judge whether their output is *correct*. A deputy can do the wrong thing for a long time before the human reads its message — quality gating is orthogonal and out of scope. (iii) *Interrupt-race correctness*. The at-least-once idempotent drain (§??) is the invariant that makes concurrent interrupts safe; a bug there reintroduces lost or duplicated feedback, so this path needs the care of any lock protocol. (iv) *Compaction fidelity*. LLM compaction can drop or distort a case despite a forbidding prompt; the mechanical fallback bounds *availability* but not *fidelity*, with the never-compacted case log as the backstop of record.

Limitations. (1) Posse targets a single-provider, single-operator deployment; the cost constants are provider-specific and multi-tenant fairness is out of scope. (2) We evaluate the design analytically and by construction, not against a controlled benchmark of other systems — a fair comparison is hard precisely because they target different regimes. (3) The continuity threshold and several policies are hand-tuned; learning them is future work. (4) Mechanical routing (I3) is less flexible than an LLM router for genuinely ambiguous messages — we accept occasional Receptionist hand-offs as the price of predictability.

13 Conclusion

Agent frameworks have largely solved the inside of a task. As deployments grow into long-running fleets, the binding constraint moves outside it: keeping many agents working, cheaply, for a long time, under a human who steers but cannot supervise. Posse is an operating system for that regime. Four decisions carry it — identity decoupled from process liveness, token cost as a scheduled resource, a zero-cost mechanical control plane separated from paid cognition, and memory as a shared record rather than

a private context — and together they turn continuity, concurrency, autonomy, cost, and cross-agent knowledge transfer from failure modes into engineered system properties. The Sheriff watches; the deputies work; the records remember; and the cost stays bounded by design.

References

- [1] Significant Gravitas. *AutoGPT*. 2023. <https://github.com/Significant-Gravitas/AutoGPT>.
- [2] Y. Nakajima. *BabyAGI*. 2023. <https://github.com/yoheinakajima/babyagi>.
- [3] S. Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR, 2023.
- [4] N. Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS, 2023.
- [5] S. Yao et al. *Tree of Thoughts*. NeurIPS, 2023.
- [6] A. Elovic et al. *GPT-Researcher: Autonomous Agent for Deep Research*. 2023. <https://github.com/assafelovic/gpt-researcher>.
- [7] J. S. Park et al. *Generative Agents: Interactive Simulacra of Human Behavior*. UIST, 2023.
- [8] Q. Wu et al. *AutoGen: Multi-Agent Conversation Framework*. arXiv:2308.08155, 2023.
- [9] G. Li et al. *CAMEL: Communicative Agents for LLM Society*. NeurIPS, 2023.
- [10] S. Hong et al. *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework*. ICLR, 2024.
- [11] C. Qian et al. *Communicative Agents for Software Development (ChatDev)*. ACL, 2024.
- [12] J. Moura et al. *CrewAI*. 2023. <https://github.com/crewAIInc/crewAI>.
- [13] LangChain. *LangGraph: Stateful, Multi-Actor LLM Applications*. 2024. <https://github.com/langchain-ai/langgraph>.
- [14] OpenAI. *Swarm: Lightweight Multi-Agent Orchestration*. 2024. <https://github.com/openai/swarm>.
- [15] Omnigent. *Omnigent: Open-Source AI Agent Framework and Meta-Harness*. 2024. <https://github.com/omnigent-ai/omnigent>.
- [16] exo. *exo: Agent Harness*. 2024. <https://github.com/exoharness/exo>.
- [17] NVIDIA. *NemoClaw: Running Agents Securely in NVIDIA OpenShell with Managed Inference*. 2025. <https://github.com/NVIDIA/NemoClaw>.
- [18] All Hands AI. *OpenHands: An Open Platform for AI Software Developers*. 2024. <https://github.com/All-Hands-AI/OpenHands>.
- [19] Langfuse. *Langfuse: Open-Source LLM Engineering and Observability*. 2023. <https://github.com/langfuse/langfuse>.

- [20] AgentOps. *AgentOps: Agent Observability and Cost Tracking*. 2024. <https://github.com/AgentOps-AI/agentops>.
- [21] Arize AI. *Phoenix: LLM/Agent Observability*. 2023. <https://github.com/Arize-ai/phoenix>.
- [22] Anthropic. *Claude Code and the Claude Agent SDK*. 2024–2026.
- [23] OpenAI. *Assistants API: Persistent Threads for Agents*. 2023–2024.
- [24] K. Mei et al. *AIOS: LLM Agent Operating System*. arXiv:2403.16971, 2024.
- [25] C. Packer et al. *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560, 2023.
- [26] Temporal Technologies. *Temporal: Durable Execution*. 2020–. <https://temporal.io>.
- [27] Amazon Web Services. *AWS Step Functions*. 2016.
- [28] M. Beauchemin et al. *Apache Airflow*. 2015.
- [29] J. Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, KTH, 2003.
- [30] C. Hewitt et al. *A Universal Modular ACTOR Formalism for Artificial Intelligence*. IJCAI, 1973.
- [31] A. Verma et al. *Large-scale Cluster Management at Google with Borg*. EuroSys, 2015.
- [32] Anthropic. *Prompt Caching (cache-control, ~5-minute TTL)*. 2024. <https://docs.anthropic.com>.
- [33] OpenAI. *Prompt Caching in the API*. 2024. <https://platform.openai.com/docs/guides/prompt-caching>.
- [34] I. Gim et al. *Prompt Cache: Modular Attention Reuse for Low-Latency Inference*. MLSys, 2024.
- [35] W. Kwon et al. *Efficient Memory Management for LLM Serving with PagedAttention (vLLM)*. SOSP, 2023.
- [36] L. Zheng et al. *SGLang: Efficient Execution of Structured Language Model Programs (RadixAttention)*. NeurIPS, 2024.
- [37] The Kubernetes Authors. *Kubernetes: Production-Grade Container Orchestration (controllers / reconciliation)*. 2014–.
- [38] A. Yoo, M. Jette, M. Grondona. *SLURM: Simple Linux Utility for Resource Management*. JSSPP, 2003.
- [39] P. Moritz et al. *Ray: A Distributed Framework for Emerging AI Applications*. OSDI, 2018.